

Étape 4

Déploiement

Une API sécurisée pour le modèle et les données, conteneurisée, testée, et une mesure de la dérive

Projet	CryptoBot, bot de trading piloté par le machine learning
Formation	Data Engineer, DataScientest
Dépôt	DataScientest-Studio/jul26_bde_crypto
Date	30 septembre 2026

Sommaire

1. Objectif de l'étape

2. Notions de base

2.1 API REST

2.2 FastAPI et Pydantic

2.3 Conteneur et image

2.4 Dérive des données

3. L'API

3.1 Routes

3.2 Principes de conception

3.3 Le carnet de positions virtuelles

4. Sécurité

4.1 Authentification par clé

4.2 La clé ne va jamais dans le navigateur

4.3 Défense en profondeur

5. Tests

5.1 Tests unitaires

5.2 Doublures

5.3 Vérification du déploiement

6. Conteneurisation

6.1 Services

6.2 Choix

7. Mesure de la dérive

7.1 L'indice PSI

7.2 Deux règles apprises

7.3 Résultat (23 septembre 2026, 90 jours)

8. L'interface

9. Annexe : fichiers

1. Objectif de l'étape

L'étape 4 fait sortir le modèle du notebook. Elle demande de l'exposer par une API, de tester cette API, de conteneuriser l'API et les bases de données, et de mesurer la dérive des données.

Demande	Réalisation	Emplacement
API du modèle et des bases	11 routes FastAPI, documentation interactive	<code>api/</code>
Tests unitaires de l'API	149 tests pytest, plus 21 vérifications du déploiement	<code>tests/</code> , <code>scripts/verifier_deploiement.py</code>
Conteneurisation	API, interface et bases dans Docker Compose	<code>Dockerfile.api</code> , <code>interface/</code> , <code>docker-compose.yml</code>
Mesure de la dérive	indice PSI par paire et par pas de temps	<code>api/derive.py</code> , <code>scripts/mesurer_derive.py</code>

2. Notions de base

2.1 API REST

Une API (interface de programmation) permet à un programme d'en appeler un autre. Une API REST passe par HTTP, le protocole du web : le client envoie une requête à une adresse (une route), avec une méthode (`GET` pour lire, `POST` pour agir), et reçoit une réponse, ici en JSON.

La réponse porte un **code de statut** qui dit comment la demande s'est passée :

Code	Signification	Exemple dans le projet
200	succès	une prédiction est rendue
401	non authentifié	clé d'API absente ou fausse
404	ressource inconnue	paire qui n'est pas suivie
422	requête invalide	style ou pas de temps mal orthographié
429	trop de requêtes	limitation de débit dépassée
503	service indisponible	base ou modèle absent

2.2 FastAPI et Pydantic

FastAPI est un framework Python pour écrire des API. Chaque route est une fonction ; ses paramètres sont déclarés avec leur type. Pydantic vérifie les données reçues avant qu'elles n'atteignent le code : un style inconnu ou un pas de temps mal écrit est refusé avec un message clair (code 422). FastAPI produit aussi automatiquement une documentation interactive (Swagger), où chaque route s'essaie depuis le navigateur.

2.3 Conteneur et image

Une **image** Docker contient une application et tout ce dont elle a besoin pour tourner : système minimal, Python, bibliothèques, code. Un **conteneur** est une image en cours d'exécution, isolée du reste de la machine. La même image donne le même comportement sur n'importe quel ordinateur, ce qui supprime le « ça marche chez moi ».

Un **volume** conserve des données en dehors du conteneur : supprimer et recréer le conteneur d'une base ne perd pas ses données. **Docker Compose** décrit plusieurs conteneurs, leurs volumes, leurs réseaux et leurs dépendances dans un seul fichier, et les démarre en une commande.

2.4 Dérive des données

Un modèle apprend sur une période. Si le marché change (volatilité qui double, volumes qui s'effondrent), les données qu'il reçoit ne ressemblent plus à celles de son apprentissage. On parle de **dérive des données** (*data drift*). Rien ne plante : le modèle continue de répondre, avec une fiabilité inconnue. D'où la nécessité de la mesurer.

3. L'API

3.1 Routes

Route	Rôle
GET /health	l'API, le modèle et les deux bases répondent-ils ? Sert de sonde à Docker
GET /modele	origine du modèle, variables, seuils, mesures
GET /paires	paires disponibles en base
GET /bougies/{paire}	dernières bougies, lues dans PostgreSQL
GET /couverture	volume et retard de collecte de chaque jeu de données
POST /prediction	acheter, vendre ou attendre, selon la paire, le pas de temps et le style
GET /derive	les données récentes ressemblent-elles à celles de l'apprentissage ?
GET /graphique/{paire}	bougies avec la décision du modèle pour chacune, pour l'interface
GET /ordre/{paire}	ordre projeté : entrée, take profit, stop loss
POST /positions/{paire}	un tour du carnet de positions virtuelles
GET /metrics	métriques pour la supervision, invisibles depuis l'extérieur

Exemple d'appel, à travers l'interface :

```
curl -X POST http://localhost:8080/api/prediction \
-H "Content-Type: application/json" \
-d '{"symbole": "BTCUSDT", "interval": "1h", "style": "conservateur"}'
```

```
{ "symbole": "BTCUSDT", "interval": "1h", "style": "conservateur",  
  "bougie": "2026-09-23T09:00:00+00:00", "probabilite_hausse": 0.5346,  
  "seuil_achat": 0.5415, "seuil_vente": 0.3921, "decision": "attendre", "sens": 0,  
  "avertissement": "modele experimental, non rentable frais compris" }
```

La probabilité (0,5346) est sous le seuil d'achat du style conservateur (0,5415) : le modèle s'abstient.

3.2 Principes de conception

- **L'API lit les vues, jamais les tables.** `v_day_trading` restitue les trois pas de temps du profil, où qu'ils soient stockés.
- **Le modèle est chargé une fois et gardé en mémoire.** Le fichier pèse une centaine de mégaoctets ; le relire à chaque appel rendrait l'API inutilisable. Quand un nouveau modèle remplace le fichier, l'API le voit à sa date de modification et le recharge sans redémarrer.
- **La bougie en cours n'est jamais prédite.** Sa clôture n'existe pas encore ; seules les bougies clôturées sont données au modèle.
- **Chaque prédiction est enregistrée** dans la table `api_predictions`, pour suivre dans le temps la répartition des décisions.
- **Les réponses portent un avertissement** : le modèle est expérimental et n'est pas rentable une fois les frais déduits.

3.3 Le carnet de positions virtuelles

Pour juger le modèle autrement que par des pourcentages, l'API tient un carnet de positions fictives. Quand le modèle donne un signal, une position est ouverte avec un take profit et un stop loss placés à trois fois la volatilité récente, et une échéance de 12 bougies. Elle se ferme à la première des trois qui est atteinte. Si le take profit et le stop loss sont touchés dans la même bougie, on retient le stop loss : les données ne disent pas lequel a été atteint en premier, on prend l'hypothèse défavorable. Une seule position est ouverte à la fois par paire, pas de temps et style.

4. Sécurité

4.1 Authentification par clé

Toutes les routes, sauf `/`, `/health` et la documentation, exigent une **clé d'API** dans l'en-tête `X-API-Key`.

Principe	Mise en œuvre
Une clé par client	l'interface et Airflow ont chacun la leur ; chaque appel est attribué à un client, et une clé compromise se révoque sans couper l'autre
Fermée par défaut	sans clé configurée, l'API répond 503 ; un oubli de configuration ne l'ouvre jamais
Comparaison en temps constant	<code>hmac.compare_digest</code> : le temps de réponse ne révèle pas combien de caractères d'une clé essayée sont justes
Clés longues et aléatoires	64 caractères hexadécimaux générés par <code>secrets</code> , stockés dans <code>.env</code> , jamais versionnés

Une clé d'API convient ici mieux qu'un jeton JWT. Le JWT répond au cas de personnes qui se connectent avec un mot de passe et reçoivent un jeton temporaire. Les clients de notre API sont des programmes, sans utilisateur derrière ; c'est le cas d'usage des clés d'API, celui de Binance, de Stripe ou de GitHub pour les accès de machine à machine.

4.2 La clé ne va jamais dans le navigateur

Tout ce qui est envoyé à un navigateur peut être lu par n'importe qui. L'interface ne contient donc aucune clé. Elle est servie par son propre conteneur, un serveur **nginx** qui joue le rôle de **proxy** : la page appelle `/api/...`, et nginx transmet la requête à l'API en y ajoutant la clé de l'interface.

4.3 Défense en profondeur

Couche	Mesure
Réseau	l'API n'a aucun port publié sur la machine ; seuls nginx, Airflow et Prometheus la joignent, dans le réseau Docker
Réseaux séparés	l'interface n'est que sur le réseau « services » : même compromise, elle ne peut pas atteindre les bases
Entrées	paires, pas de temps et styles vérifiés par liste blanche, avant tout appel à Binance
Débit	nginx limite chaque adresse à 10 requêtes par seconde (rafales de 40), et répond 429 au-delà
Navigateur	en-têtes de sécurité : politique de contenu, interdiction d'afficher la page dans un cadre
Métriques	<code>/api/metrics</code> renvoie 404 depuis l'extérieur
Conteneurs	l'API et l'interface tournent sans droits administrateur
Ports	tous les ports publiés sont liés à <code>127.0.0.1</code> : joignables depuis la machine, jamais depuis le réseau

5. Tests

5.1 Tests unitaires

Un **test unitaire** vérifie un comportement précis, automatiquement. Les 149 tests du projet s'exécutent en une dizaine de secondes avec `pytest tests/ -q`.

Famille	Nombre	Ce qu'elle vérifie
API	59	chaque route, les décisions des deux styles, les codes d'erreur, le carnet de positions
dont sécurité et validation	16	refus sans clé, fausse clé, API fermée par défaut, liste blanche, et une vérification de toutes les routes
Pipeline et supervision	23	contrôle qualité, ingestion, règle de promotion d'un modèle, rechargement à chaud
Variables	4	pas de regard vers le futur, indépendance de la fenêtre de calcul et du niveau de prix
Étiquetage	19	barrières, absence de fuite temporelle
Données et configuration	44	nettoyage, cohérence entre le code et le schéma SQL

Un des tests parcourt toutes les routes de l'API et vérifie qu'elles refusent un appel sans clé : une route ajoutée sans protection le fait échouer.

5.2 Doublures

Aucun test ne dépend des bases ni du vrai modèle. Les accès extérieurs sont remplacés par des **doublures** : de fausses bougies, un faux modèle qui renvoie une probabilité choisie. Trois raisons : les tests passent sur n'importe quelle machine sans Docker, ils durent quelques secondes, et un échec désigne un bug du code et non une base éteinte. Par exemple, en réglant le faux modèle sur 0,58, on vérifie que le style agressif achète et que le conservateur s'abstient.

5.3 Vérification du déploiement

Ce qui n'existe qu'une fois les conteneurs assemblés ne peut pas être vérifié par un test unitaire : la clé ajoutée par nginx, l'API invisible de l'extérieur, la limitation de débit. Le script `scripts/verifier_deploiement.py` le vérifie en appelant l'application démarrée comme le ferait un utilisateur. Il fait 21 vérifications, dont une rafale de 120 appels : environ 75 sont refusés par nginx.

6. Conteneurisation

6.1 Services

Service	Image	Accès depuis la machine
postgres	TimescaleDB (PostgreSQL 16)	127.0.0.1:5432
mongo	MongoDB 7	127.0.0.1:27017
api	image du projet (<code>Dockerfile.api</code>)	aucun
interface	nginx sans droits administrateur	localhost:8080
pgadmin, mongo-express	outils de consultation	127.0.0.1:5050, 127.0.0.1:8081

L'étape 5 ajoute Airflow, MLflow et la supervision dans le même fichier. Les services sont rangés en **profils** Compose (`pipeline`, `supervision`, `outils`) que l'on active ou non ; le produit (bases, API, interface) démarre toujours.

6.2 Choix

- **Une image légère pour l'API.** Elle n'a besoin ni de MLflow, ni de Jupyter, ni de matplotlib : ses dépendances sont listées à part (`requirements-api.txt`).
- **Le modèle n'est pas dans l'image.** Il est monté depuis `./models` en lecture seule : un nouveau modèle se met en service sans reconstruire l'image.
- **Des sondes de santé.** Chaque service vérifie qu'il répond vraiment ; ceux qui en dépendent attendent qu'il soit sain avant de démarrer, et Docker redémarre ce qui cesse de répondre.
- **Des secrets générés.** `python -m scripts.generer_secrets` crée le fichier `.env` avec des clés et des mots de passe aléatoires, sans jamais remplacer une valeur existante.

```
python -m scripts.generer_secrets    # une fois
docker compose up -d --build         # tout démarre
```

7. Mesure de la dérive

7.1 L'indice PSI

Pour chaque variable, on photographie la répartition de ses valeurs pendant l'apprentissage, découpée en 10 tranches de même effectif (les déciles). On regarde ensuite comment se répartissent les valeurs récentes dans ces mêmes tranches. Si rien n'a changé, chaque tranche reçoit 10 % des bougies.

L'écart se résume par l'indice de stabilité de population (**PSI**) :

```
PSI = somme sur les tranches de (p_récent - p_référence) × ln(p_récent / p_référence)
```

PSI	Lecture	Action
moins de 0,10	stable	rien à faire
0,10 à 0,25	dérive modérée	surveiller
plus de 0,25	dérive forte	réentraînement à envisager

7.2 Deux règles apprises

1. **Comparer des choses comparables.** La référence est calculée par paire et par pas de temps. La taille moyenne d'un trade vaut 0,005 sur le bitcoin et 115,8 sur le XRP ; l'amplitude d'une bougie 4 h vaut quatre fois celle d'une bougie 15 minutes. Une référence mélangée annonçait une dérive massive alors que rien n'avait bougé.

2. Une fenêtre en jours, identique pour chaque pas de temps. La première version du script lisait assez de bougies pour couvrir 90 jours en 15 minutes, mais sans couper les autres pas de temps : le 1 h était mesuré sur 360 jours et le 4 h sur deux ans, ce qui masque tout changement récent. La fenêtre est désormais de 90 jours pour chacun.

7.3 Résultat (23 septembre 2026, 90 jours)

Paire	15 min	1 h	4 h	Variables les plus concernées
BTCUSDT	3 fortes	5 fortes	7 fortes	volatilité du contexte 4 h
ETHUSDT	9 fortes	9 fortes	12 fortes	volatilité, à toutes les échelles
BNBUSDT	7 fortes	7 fortes	8 fortes	volatilité, taille moyenne d'un trade
SOLUSDT	11 fortes	10 fortes	14 fortes	taille moyenne d'un trade, volatilité
XRPUSDT	7 fortes	6 fortes	8 fortes	taille moyenne d'un trade, volatilité

Toutes les séries dérivent. Le marché des trois derniers mois ne bouge pas avec la même amplitude que pendant l'apprentissage du modèle (fin août 2024 à début février 2026). La taille moyenne d'un trade, seule variable qui dépende de l'unité de la paire, est la première candidate à une révision. L'étape 5 montre qu'une dérive des données ne s'accompagne pas forcément d'une baisse de performance.

8. L'interface

L'interface (`localhost:8080`) affiche les bougies en direct, la décision du modèle et le carnet de positions : une flèche à chaque ouverture, un point à chaque fermeture avec son gain net, les zones de take profit et de stop loss en rectangles. Le style se change dans la page et l'historique remonte jusqu'à un an. Le graphique utilise la bibliothèque libre Lightweight Charts, livrée avec le projet plutôt que chargée depuis Internet.

9. Annexe : fichiers

Fichier	Rôle
api/main.py	routes FastAPI
api/securite.py	clés par client, vérification
api/modele.py	chargement et rechargement du modèle, prédiction
api/positions.py	carnet de positions virtuelles
api/derive.py	calcul du PSI
interface/	page, configuration nginx, image
Dockerfile.api, docker-compose.yml	images et services
tests/	tests unitaires
scripts/verifier_deploiement.py	vérification de l'application démarrée